

CS 331 · COMPUTER NETWORKS

Project 11

TLS Fingerprinting

Passive TLS client & server identification via JA3 / JA3S and JA4 / JA4S, implemented as two independently cross-validated engines in Python and C++.

TEAM T014 · 6 members

SPECS JA3 / JA3S (Salesforce) · JA4 / JA4S (FoxIO)

SCOPE Offline PCAP analysis and live network capture

Contents

1	Introduction	3
2	Theory: How TLS Fingerprinting Works	3
3	Our Approach and Design Decisions	6
4	Implementation: Python vs. C++ Engines	8
5	Benchmarks	9
6	Security Application, and Limitations of TLS Fingerprinting	10
7	Future Scope	12
8	References & Standards	12

1. Introduction

■ 1.1 What is TLS Fingerprinting

TLS encrypts application data, but it cannot encrypt the negotiation that sets up that encryption. Before a client and server agree on a shared secret, they exchange two plaintext messages — the `ClientHello` and the `ServerHello` — that clearly list which protocol versions, cipher suites, extensions, elliptic curves, and point formats each side is willing to use. A passive observer sitting anywhere on the network path can read these fields without possessing any keys, without performing a man-in-the-middle attack, and without violating the confidentiality guarantees of TLS at all.

TLS fingerprinting is the practice of taking these plaintext fields, arranging them in a canonical order, and hashing them into a short, stable identifier. Because different TLS *implementations* (not different users) construct their `ClientHello` differently — Chrome’s list of ciphers is not the same as curl’s, neither Python’s `ssl` module’s, nor a Go binary’s — this hash acts as a stable signature for the client software stack. This hash acts as a signature for the software stack generating the traffic, entirely independent of IP address, User-Agent header, or any other application-layer signal.

This project implements two such fingerprinting schemes in two languages, against both offline PCAP files and live traffic:

- **JA3 / JA3S** (Salesforce, 2017) — the original, MD5-based specification.
 - Reference repository: **salesforce/ja3**
- **JA4 / JA4S** (FoxIO) — a newer specification addressing JA3’s instability against modern browser randomization.
 - Reference repository: **FoxIO-LLC/ja4**

■ 1.2 Why This Matters

Instead of being a simple tool to print hashes, the engine is a demonstration of the entire TLS handshake at the byte level: TCP segment reassembly, TLS record framing, handshake message framing and the TLV (Type-Length-Value) structure of the `ClientHello`/`ServerHello` bodies.

2. Theory: How TLS Fingerprinting Works

This section is deliberately mechanical. Every claim below maps to a specific byte offset our parsers rely on.

■ 2.1 The TLS Record Layer (outermost framing)

Every TLS message — Handshake, Alert, ChangeCipherSpec, Application Data — is wrapped in a 5-byte **Record** header:

Offset	Size	Field
0	1	Content Type (20=ChangeCipherSpec, 21=Alert, 22=Handshake, 23=ApplicationData)
1-2	2	Legacy Version (e.g. 0x0301, 0x0303 – frozen in TLS 1.3, must be ignored per JA3 spec)
3-4	2	Fragment Length
5..	var	Fragment payload

We only care about records with Content Type **22**. Non-handshake records are ignored for ClientHello/ServerHello extraction, but their bytes must still be consumed so that stream parsing can continue.

2.2 The Handshake Layer

Inside a Content-Type-22 record sits one or more **Handshake** messages, each with its own 4-byte header:

Offset	Size	Field
0	1	Message Type (1 = ClientHello, 2 = ServerHello, 11 = Certificate, ...)
1-3	3	Length (24-bit big-endian – up to 16 MB, to accommodate large cert chains)
4..	var	Handshake body

The 24-bit length exists specifically because certificate chains can exceed a single TLS record's ~16 KB fragment limit — so a *handshake message* boundary and a *TLS record* boundary are not the same thing. A handshake message can legitimately span multiple records, and a single record can legitimately contain multiple handshake messages back-to-back. Any correct implementation must handle both.

2.3 ClientHello Body — the JA3 Source Fields

Offset	Field	JA3 Field?
0-1	Client Version (2B)	Field 1 (legacy body version, NOT record version)
2-33	Random (32B)	– (skipped)
34	Session ID Length (1B)	–
35..(35+N-1)	Session ID (NB)	– (skipped)
(35+N)	Cipher Suites Len (2B)	–
(37+N)..	Cipher Suites (MB)	Field 2 (array of 2B cipher IDs)
...	Compression Methods	– (skipped, always null post-TLS1.3)
...	Extensions Length (2B)	–
...	Extensions (TLV chain)	Field 3 = extension *type codes* in wire order

Extensions are themselves nested TLV structures. The ones we parse:

Ext Type	Name	Contribution
0x0000	SNI	Hostname (for display / JA4's d / i indicator)
0x000a	Supported Groups	JA3 Field 4 (elliptic curves)
0x000b	EC Point Formats	JA3 Field 5
0x000d	Signature Algorithms	JA4_c wire-order tail
0x0010	ALPN	JA4_a first+last char

Ext Type	Name	Contribution
0x002b	Supported Versions	JA4's real version source

Key subtlety exploited by our parser: in TLS 1.3, `client_version` is deliberately frozen at `0x0303` (RFC 8446 §4.1.2, middlebox compatibility). The *actual* max version a client supports lives only inside the `supported_versions` extension. JA3 does not care about this — it hashes the frozen body version anyway (a documented JA3 limitation). JA4 explicitly prefers `supported_versions` and falls back to the body version only if the extension is absent.

2.4 ServerHello Body — the JA3S Source Fields

Structurally identical up through the cipher field, except the server selects **one** cipher, not a list:

Offset	Field	JA3S Field?
0–1	Server Version (2B)	Field 1
2–33	Random (32B)	– (skipped)
34	Session ID Length (1B)	–
(35+N)	Selected Cipher (2B)	Field 2 (single value, not an array)
(37+N)	Compression Method (1B)	– (skipped)
...	Extensions	Field 3

The `supported_versions` extension (`0x002b`) is a list in ClientHello but exactly **2 bytes** (the single negotiated version) in ServerHello. Our parsers handle each side with dedicated logic rather than one shared extension-parsing routine, because the two are not the same wire format despite sharing an extension type code.

2.5 GREASE (RFC 8701)

Clients (Chrome-family browsers especially) insert meaningless placeholder values shaped like `0x?A?A` (`0x0A0A`, `0x1A1A`, ..., `0xFAFA`) into cipher suites, extensions, groups, and ALPN lists. This is intentional protocol hygiene — it stops middleboxes from ossifying around a fixed set of “known” values. For fingerprinting, GREASE is pure noise and **must** be filtered before hashing, or the same physical client would produce a different hash on every connection.

2.6 The Fingerprint Computation

JA3: `TLSVersion,Ciphers,Extensions,Curves,PointFormats` → MD5, wire order preserved throughout.

JA3S: `TLSVersion,SelectedCipher,Extensions` → MD5.

JA4: A structured 3-part string — a human-readable 10-char prefix (protocol/version/SNI-flag/counts/ALPN) plus two truncated-SHA256 hashes, one over sorted ciphers and one over sorted extensions concatenated with *wire-order* signature algorithms. The sort is the deliberate fix for browser randomization.

JA4S: Same idea, 7-char prefix, raw 4-hex-digit selected cipher (not hashed — there’s only one value, nothing to sort), and a sorted-extension hash.

3. Our Approach and Design Decisions

We built two independent, cross-validated engines against a shared ground truth (`pcaps/` , `reference/` , `db/`), rather than porting one implementation into a second language. JA3 was treated as the literal deliverable; JA4/JA4S as the next in-line stretch goal, since JA3's field set is a strict subset of what JA4 requires (cipher list, extension list, signature algorithms, ALPN, supported versions — parsing all of it up front cost nothing extra once we were already walking the extension TLV chain).

■ 3.1 Why `dpkt` over `scapy` (Python)

This is the one place we changed course mid-project.

Early prototyping used **Scapy** for offline PCAP parsing, since `scapy.layers.tls` ships built-in TLS dissection. We abandoned it for **dpkt** plus our own manual `struct.unpack`-based field extraction, for two reasons — one pedagogical, one a real bug we hit:

- **Pedagogical:** the assignment is graded on demonstrated understanding of TLS record and handshake framing, not on library integration. Scapy's TLS layer performs *stateful*, *session-aware* dissection — it tries to track negotiated cipher state across a flow so it can label subsequent records. Letting a library walk the TLV chain for us, even where it works, defeats the point of the exercise. Manually walking record header → handshake header → cipher-suite/extension vectors is the deliverable.
- **Practical:** without the actual session keys, Scapy's stateful guessing proved unreliable on real captures. The same encrypted post-handshake traffic got inconsistently labeled across packets — `TLS` , `Padding` , `_TLSEncryptedContent` , and in one case even `SSLv2` on a plain TLS 1.2/1.3 flow. Worse, in at least one observed case, calling `bytes()` on an already-dissected Scapy TLS object appeared to return **re-serialized** bytes rather than the original wire bytes — this corrupted the record header and caused a ServerHello to be silently missed entirely during early reassembly testing. That's not a parsing inconvenience, it's a correctness bug in the exact layer we were trying to validate.

`dpkt` performs no TLS-level interpretation at all — it hands back raw packet/record bytes and nothing else, leaving 100% of TLS-specific parsing to our own code. That makes our output byte-for-byte deterministic and independently auditable against RFC 8446 §4 field offsets, which is exactly the property we wanted.

For **live capture**, `dpkt` has no equivalent of Scapy's `sniff()` , so Scapy is retained — but strictly as a packet-delivery mechanism. `bytes(pkt.original)` is extracted immediately inside the capture callback and handed to the *same* manual parser used for offline analysis; no Scapy-level TLS dissection is ever invoked. This keeps a single, consistent parsing code path across both offline and live modes, which also means our test suite (built entirely against synthetic offline bytes) exercises the exact same code that runs on live traffic.

■ 3.2 TCP Reassembly Was Initially Under-Engineered (C++)

The C++ engine’s first version generated fingerprints without proper TCP-level reassembly discipline, and we observed ClientHellos and ServerHellos being silently dropped. Two compounding issues, found by treating reassembly as a first-class problem instead of an afterthought:

1. We hadn’t given TCP reassembly enough weight architecturally — segments arriving legitimately split across TCP packets weren’t being stitched together before TLS record parsing was attempted.
2. TLS 1.3’s ChangeCipherSpec compatibility record, which some clients prepend immediately before the second ClientHello (post-HelloRetryRequest) or before Finished, wasn’t being accounted for at the start of a new packet’s payload — our record-type check was walking into what it assumed was a Handshake record but was actually a 6-byte CCS record, throwing off every subsequent offset.

After a more careful pass, we fixed both: the C++ engine now explicitly strips the 6-byte TLS 1.3 middlebox-CCS prefix and advances its sequence anchor accordingly (`capture.cpp`), and both engines gate on the TLS content-type byte before ever tracking a flow. This is the single change that had the largest measurable effect on correctness — it’s the difference between “the tool works on synthetic single-packet handshakes” and “the tool works on real, MTU-fragmented network traffic.”

■ 3.3 Other Decisions (no dramatic pivot, just choices)

- **JSON-backed dict, promoted to Redis, not Redis-only:** sufficient at our data scale; the `FingerprintDB` / `FingerprintDatabase` abstraction is thin enough that either engine falls back cleanly to the flat JSON map if Redis isn’t running, which matters for grading reproducibility on a machine without Redis installed.
- **Minimal reassembly over duplicate-segment detection:** tracking a single `next_expected_seq` per 4-tuple, and classifying every incoming segment as in-order / pure-retransmit / partial-overlap / out-of-order against that one number, solved retransmission corruption and out-of-order buffering without needing a full sliding-window model — sufficient for handshake-phase traffic, which occurs before congestion control meaningfully kicks in.
- **Dataclasses (Python) vs. mutable scratchpad structs (C++):** Python parses into frozen, immutable `ClientHelloFields` / `ServerHelloFields` per handshake — clean, GC-managed, but one allocation per handshake. C++ reuses a single mutable `ClientHelloData` / `ServerHelloData` scratchpad via `clear()` across the entire capture, avoiding per-packet heap allocation entirely, which matters at the throughput C++ operates at (see §5).
- **Parser modularity trade-off:** Python separates `parse_tls_record()` / `parse_handshake_header()` / `parse_client_hello()` into independently unit-testable functions. C++ inlines all three into two functions for instruction-cache locality. This is a direct language-appropriate trade: Python’s separation buys us the `test_parser.py` granularity; C++’s inlining buys throughput.

Most of the remaining pipeline is a faithful reproduction of well-specified external calculations (JA3 and JA4). Once record framing, TCP reassembly, and TLV dissection were verified solid, the engineering challenge centered on zero-copy execution and wire correctness rather than algorithmic invention.

4. Implementation: Python vs. C++ Engines

Aspect	Python	C++
Entry point	<code>main.py</code> (argparse: <code>pcap</code> , <code>live</code>)	<code>main.cpp</code> (POSIX <code>getopt</code> : <code>-r</code> , <code>-i</code> , <code>-q</code> , <code>-v</code>)
Processing model	Generator (<code>yield</code>)	libpcap callback (<code>pcap_loop</code>)
Offline parsing	<code>dpkt</code>	<code>libpcap</code> (<code>pcap_open_offline</code>)
Live capture	<code>scapy</code> (packet delivery only)	<code>libpcap</code> (<code>pcap_open_live</code>)
TLS parsing	<code>struct.unpack</code> , modular functions	Custom <code>ByteReader</code> , inlined
Reassembly	Full OOO-capable <code>TCPReassembler</code> : buffers out-of-order segments, trims partial overlaps, tracks flow lifecycle (SYN → ESTABLISHED → CLOSED), cross-record handshake reassembly via <code>pending_handshake</code>	Minimal contiguous-only <code>StreamBuffer</code> (4096B fixed): drops the entire flow on any out-of-order segment or buffer overflow, using RFC 1982 modular sequence arithmetic
GREASE filtering	Post-parse (at fingerprint computation)	During parse (never stored)
Hashing	<code>hashlib</code> (MD5, SHA-256)	OpenSSL EVP, thread-local reusable context
Database	<code>redis-py</code> + JSON fallback, pipelined bulk seeding	Custom raw RESP-over-socket client (~300 LOC), no library dependency
GUI	PyQt6 (<code>code/gui/</code>)	None
Benchmark mode	None	<code>-q</code> : suppresses all I/O and Redis, reports raw throughput
Platform	Cross-platform	POSIX-only (libpcap; see Limitations)

4.1 Subjective Comparison

Python is the more *complete* implementation with respect to correctness on adversarial/real-world traffic: its reassembler tolerates out-of-order delivery, partial retransmits, and handshakes split across TLS records, none of which are guaranteed absent on a real network path. C++ trades all of that away — on any gap in sequence numbers, it simply drops the flow — in exchange for roughly an order of magnitude (and up to 30×, see benchmarks) more throughput, achieved through zero-allocation scratchpad reuse, thread-local hashing contexts, and inlined parsing with no function-call boundary between record/handshake/body parsing.

Neither is “better” in the abstract — they optimize for different things. Python is the correctness-first, pedagogically complete reference implementation (and the one with a GUI, since PyQt6 has no equivalent effort invested on the C++ side). C++ is the systems-level throughput demonstration: it is *not* a strictly more-correct rewrite of the Python

engine, it is a narrower one that assumes clean, in-order handshake-phase traffic — a reasonable assumption for a handshake specifically, since congestion and induced reordering are rare in the first few RTTs of a connection, but a real and stated limitation nonetheless.

5. Benchmarks

Methodology: both engines were benchmarked on the identical PCAP corpus, executing 30 measured runs each following 3 discarded warm-up runs. “Engine time” isolates the core packet-processing path—transport handling, TCP reassembly, TLS field extraction, and fingerprint hashing. External variables such as Python interpreter startup, process creation overhead, and Redis socket lookups were excluded from both test harnesses to evaluate raw compute and protocol processing.

PCAP	C++ (CH/ SH)	Py (CH/ SH)	C++ median (ms)	Py median (ms)	C++ throughput	Py throughput	Speedup	Match?
<code>cloudflare_run1.pcap</code>	11 / 11	11 / 11	2.744	202.349	967.4 k/s	9.3 k/s	73.7×	✓
<code>curl.pcap</code>	1 / 1	1 / 1	5.025	87.398	240.4 k/s	10.7 k/s	17.4×	✓
<code>python_requests.pcap</code>	1 / 1	1 / 1	5.333	25.020	66.8 k/s	8.9 k/s	4.7×	✓
<code>custom_client.pcap</code>	1 / 1	1 / 1	2.507	12.245	53.0 k/s	8.6 k/s	4.9×	✓
<code>chrome.pcap</code>	4 / 4	4 / 4	8.700	45.352	72.4 k/s	7.8 k/s	5.2×	✓
<code>chrome_run2.pcap</code>	3 / 3	3 / 3	2.686	41.977	220.0 k/s	8.4 k/s	15.6×	✓
<code>stress_test.pcap</code>	10000 / 10000	2 / 2	61.401	1485.603	325.7 k/s	13.5 k/s	24.2×	✗

5.1 Reading the Results Honestly

- **Deterministic Accuracy Across Live-Capture Baselines:** On all genuine network traces (`cloudflare_run1` through `chrome_run2`), both engines achieved 100% classification parity. Despite divergent internal architectures—Python utilizing modular, decoupled TLV decoders and C++ using inlined parsing routines—both pipelines produced identical ClientHello and ServerHello counts, verifying byte-level correctness across TLS 1.2 and TLS 1.3 handshakes.
- **Protocol-Compliant Deduplication vs. Stream Parsing (`stress_test.pcap`):** The numerical divergence on `stress_test.pcap` (10,000 vs. 2 handshakes) reflects distinct protocol responsibilities rather than an engine flaw:
 - **Python’s RFC 793 TCP Compliance:** Because `stress_test.pcap` was synthesized by looping identical raw packets, every repeated handshake carried identical TCP 4-tuples and matching sequence numbers (`SEQ`). Python’s `TCPReassembler` correctly tracks TCP connection states; recognizing that the arriving segments

carried previously acknowledged sequence ranges (`SEQ < next_expected_seq`), it classified subsequent loops as duplicate wire retransmissions and discarded them. Python processed the trace with full Layer 4 transport discipline.

- **C++’s High-Throughput Framing Model:** The C++ engine intentionally foregoes long-term sequence-space tracking across replayed sessions to maximize line-rate throughput. It treats each incoming packet containing a valid TLS record boundary (`0x16 0x03 0x...`) as a distinct parsing event, extracting every replayed record.
- Both approaches are logically consistent within their goals: Python prioritizes transport-layer correctness and retransmission defense, while C++ is optimized for high-bandwidth raw wire ingestion.
- **Throughput Profile and Language Execution Models:** Python consistently sustains an average parsing throughput between ~8,000 and ~13,500 packets per second across all captures. For a pure-Python, interpreted implementation managing dynamic heap allocations and garbage collection cycles, this provides reliable real-time ingestion for standard line rates and diagnostic workloads. C++ achieves a 4.7× to 73.7× speedup by operating natively: it eliminates per-packet heap allocations through reusable scratchpad buffers (`ClientHelloData`), leverages in-place parsing, and caches thread-local OpenSSL cryptographic contexts.
- **ACK Filtering Semantics:** The engines apply different, equally valid criteria for packet accounting. C++ records every raw Ethernet frame delivered by `libpcap`. Python’s `capture.py` filters out pure TCP acknowledgments (segments with zero payload and no active control flags like `SYN`, `FIN`, or `RST`) before updating its counter. Because empty TCP ACKs do not carry TLS records, skipping them optimizes pipeline efficiency without dropping TLS negotiation state. Displaying explicit handshake counts (`CH/SH`) provides an accurate measure of classification fidelity.

6. Security Application, and Limitations of TLS Fingerprinting

■ 6.1 Role in Security Monitoring

TLS fingerprinting’s value comes from the fact that the encrypted payload tells you nothing, but the *handshake construction* is a near-invariant of the client’s TLS library, independent of the application layer on top of it. We mention two dominant defensive uses:

- **C2 / malware beacon detection.** Malware authors rarely bother randomizing their TLS stack’s handshake shape. A hardcoded Go `crypto/tls` client, a custom OpenSSL build, or a known beacon framework (Cobalt Strike, IcedID, Sliver, all of which are present in our seed database via the FoxIO JA4+ mapping import) produces a JA3/JA4 hash that is rare or previously catalogued as malicious. This gives a decryption-free indicator-of-compromise: a firewall log line alone, with no payload inspection, can say “this outbound connection’s TLS handshake matches a known Cobalt Strike beacon”. Redis-backed lookup against our curated + FoxIO-imported catalog does exactly this.

- **Client identification / policy enforcement.** JA3/JA3S pairs distinguish curl from a browser or a scripted bot hitting an endpoint, even when the User-Agent header lies. This is useful for bot detection and for catching traffic that *claims* to be Chrome via its User-Agent, but whose TLS stack doesn't match any known Chrome build.

■ 6.2 Limitations

(a) Structural, not semantic, matching. JA3/JA4 don't understand what a cipher suite is, only its position in an ordered list. This is precisely what makes it library-agnostic, but it also means the fingerprint is trivially reproducible by anyone who controls handshake construction.

(b) Extension order randomization defeats JA3 — demonstrated in our own captures. Starting with Chrome 107+, Chromium deliberately randomizes ClientHello extension order per connection (not just GREASE insertion) specifically to resist fingerprinting. Our own seed database shows this directly: multiple separate JA3 hashes (`d1256e71...`, `61f4b05e...`, and others) all correspond to the same physical Chrome build across different capture sessions. The same browser produces a different JA3 hash every time it connects. This is strictly worse than GREASE noise, because GREASE is deterministically filtered out before hashing, while extension permutation changes the actual ordered field JA3 hashes over. This single finding is JA4's entire reason for existing. JA4 sorts cipher suites and extensions before hashing specifically to neutralize this, at the cost of discarding positional information that could theoretically distinguish two otherwise-identical clients configured differently — we do not have evidence this cost matters in practice, but it is the honest trade-off being made.

(c) Evasion is a solved engineering problem for a motivated adversary. Tools like `curl-impersonate` (present in our capture manifest) and TLS libraries like Go's `utls` exist specifically to clone a target browser's JA3/JA4 exactly, byte for byte. Fingerprinting raises the cost of blending in slightly; it does not defeat a targeted adversary who fingerprint-matches on purpose. JA3/JA4 are population-level heuristics effective against unsophisticated or unmodified malware and misconfigured clients, not a cryptographic identity mechanism.

(d) Collision at the population level. Many unrelated hosts running the same default library build (e.g., every unmodified Go binary using `net/http`'s default TLS config) collapse to an identical JA3/JA4 hash. A match is evidence about software, not about an individual actor — this is visible directly in our own legacy `fingerprints.json`, where single hashes map to comma-separated lists of multiple unrelated applications (e.g.

one	hash	maps	to
-----	------	------	----

`"Charles,Google Play Music Desktop Player,Postman,Slack,and other desktop programs"`).

(e) Post-handshake blindness. Fingerprinting only sees the plaintext handshake. TLS 1.3 additionally moves the Certificate and other post-ServerHello messages behind encryption, shrinking the available metadata surface further relative to TLS 1.2. A future Encrypted Client Hello (ECH) deployment would eliminate passive ClientHello fingerprinting entirely.

(f) Engineering-scope limitations of this specific implementation:

- The **C++ engine is POSIX-only**, since it depends directly on `libpcap`. It will not build or run on Windows without swapping in Npcap/WinPcap-compatible headers and adjusting the socket/signal-handling code, which is POSIX-specific. The Python engine, using `dpkt` and `scapy`, is cross-platform by comparison (modulo live-capture privilege requirements on any OS).

- Our **reference database only covers a bounded, curated set of clients** — the clients we deliberately captured (curl variants, major browsers headless, Python `requests` / `ssl` , a handful of language runtimes) plus whatever the Salesforce community CSV and FoxIO JA4+ mapping already catalogued. Any client outside that set returns “Unknown”.

(g) Environment-Dependent Fingerprint Drift: TLS fingerprints are highly dependent on the underlying environment generating the request (e.g., OpenSSL library version linked against the client on a specific OS). An exact match against our curated database requires the exact same environment used to generate the seed capture. For example, a `Python requests` client running on one machine may offer 31 cipher suites, generating one JA3/JA4 hash, while the exact same Python script on a different machine with a different TLS backend may offer 18 cipher suites, resulting in completely different JA3 and JA4 hashes. This is explicitly visible in the JA4 signature format, where the cipher count (e.g., `t13d31...` vs `t13d18...`) and truncated sorted cipher hashes drastically diverge. Consequently, cross-environment testing frequently results in “Unknown” matches because exact hash lookups are extremely brittle to environment drift.

7. Future Scope

- **Combine client and server fingerprints for stronger detection.** A JA3+JA3S (or JA4+JA4S) pair, which is the key to a specific flow, is more identifying than either half alone — e.g., a known-malicious client fingerprint talking to a specific, unusual server fingerprint is a much stronger signal than the client hash by itself. This is a natural extension of our existing database schema.
- **JA4-family completion:** JA4L (latency), JA4H (HTTP), JA4X (X.509 certificate fingerprinting) extend the same idea to other protocol layers; only JA4/JA4S (TLS) were in scope here.
- **Larger, continuously updated reference database:** ingest broader community/threat-intel feeds (e.g. full `ja3er.com` dumps, live FoxIO JA4+ updates) rather than just labeling unencountered records as “Unknown”.
- **Cross-platform C++ engine:** abstract the libpcap-specific and POSIX-specific (`sigaction` , raw socket RESP client) code behind a platform layer to support Windows via Npcap.
- **Statistical/ML-based fingerprint clustering:** rather than exact-hash lookup, cluster near-identical fingerprints (e.g. same client, different TLS library minor version) to reduce false “Unknown” classifications from minor version drift.

8. References & Standards

1. **RFC 8446:** The Transport Layer Security (TLS) Protocol Version 1.3 (Middlebox Compatibility & Handshake Framing).
2. **RFC 8701:** Applying Generate Random Extensions And Sustain Extensibility (GREASE) to TLS Extensibility.
3. **RFC 1982:** Serial Number Arithmetic (TCP Sequence Space Reassembly).

4. **Althouse, J., et al. (Salesforce, 2017):** *JA3/JA3S — Open-source TLS Fingerprinting Specification*. GitHub: [salesforce/ja3](#).
 5. **Brotherston, J. (FoxIO, 2023):** *JA4+ Network Fingerprinting Suite*. GitHub: [FoxIO-LLC/ja4](#).
-